

# 最短路径问题

## 《算法设计与分析》

信息科学与技术学院 张德富



# 02



## 单个源点的最短路径问题



# 一、单个源点的最短路径问题

---

## ■ 1 BellmanFord 算法

给定一个源点为的带权有向图 $G = (V, E)$ ，带权的含义同前。BellmanFord算法对图中的每条边进行松弛，最后返回一个布尔值，它表示是否有一条从源点 $s$ 可达的回路，回路的总权值为负，这个回路也称负权回路。如果有一条这样的回路，则算法回答问题没有解；若没有这样的回路，则算法产生所有从源点可达顶点的路径以及路径的权值。

**BellmanFord( $G, w, s$ )**

**1 InitializeSingleSource( $G, s$ )**

**2 for  $i \leftarrow 1$  to  $|V| - 1$  do**

**3   for each edge  $(u, v) \in E$  do**

**4     Relax( $u, v, w$ )**

**5 for each edge  $(u, v) \in E$  do**

**6   if  $d[v] > d[u] + w(u, v)$  then return False**

**7 return True**

**$\Theta(|V|)$**

**$O(|V||E|)$**

**$O(|E|)$**

**The running time  $O(|V||E|)$**

# An example

**BellmanFord( $G, w, s$ )**

**1 InitializeSingleSource( $G, s$ )**

**2 for  $i \leftarrow 1$  to  $|V| - 1$  do**

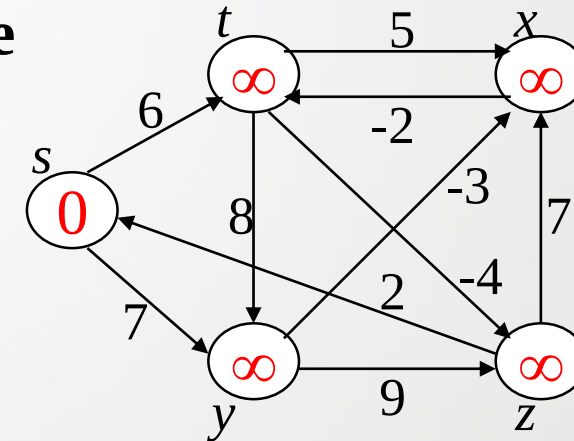
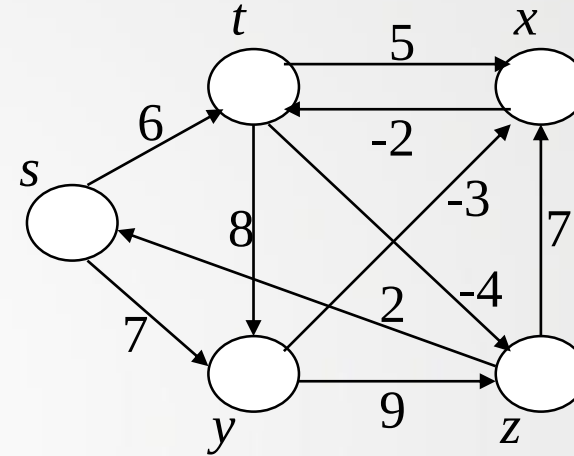
**3   for each edge  $(u, v) \in E$  do**

**4     Relax( $u, v, w$ )**

**5 for each edge  $(u, v) \in E$  do**

**6   if  $d[v] > d[u] + w(u, v)$  then return False**

**7 return True**



**BellmanFord( $G, w, s$ )**

**1 InitializeSingleSource( $G, s$ )**

**2 for  $i \leftarrow 1$  to  $|V| - 1$  do**

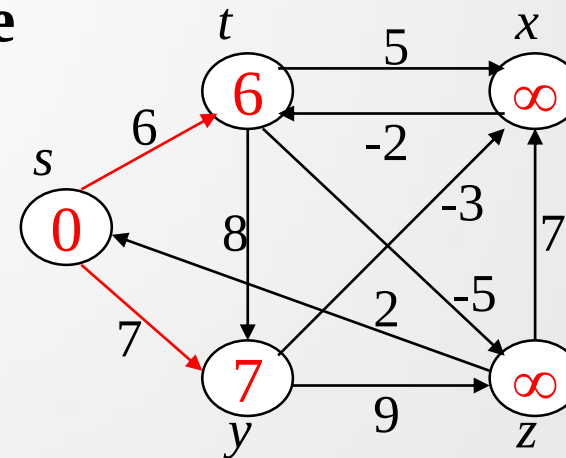
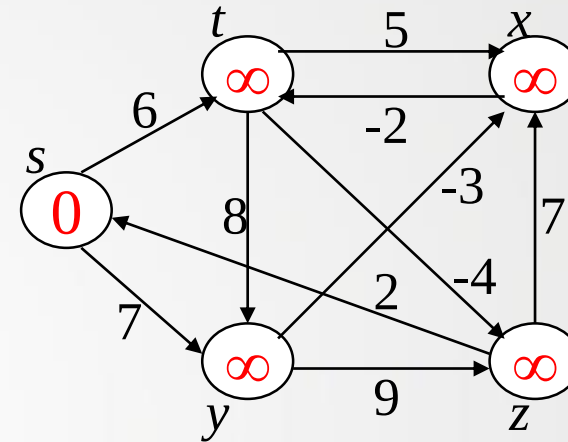
**3   for each edge  $(u, v) \in E$  do**

**4     Relax( $u, v, w$ )**

**5 for each edge  $(u, v) \in E$  do**

**6   if  $d[v] > d[u] + w(u, v)$  then return False**

**7 return True**



**BellmanFord( $G, w, s$ )**

**1 InitializeSingleSource( $G, s$ )**

**2 for  $i \leftarrow 1$  to  $|V| - 1$  do**

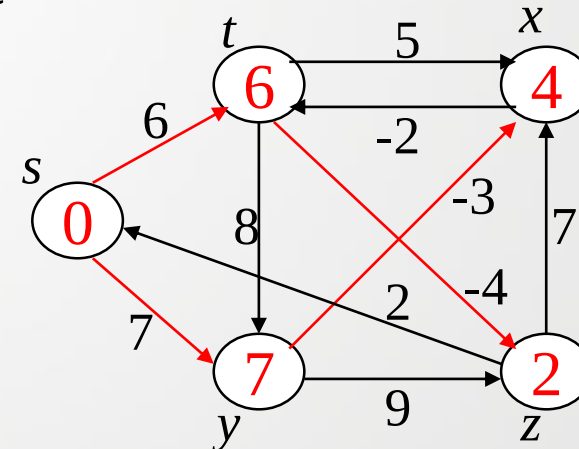
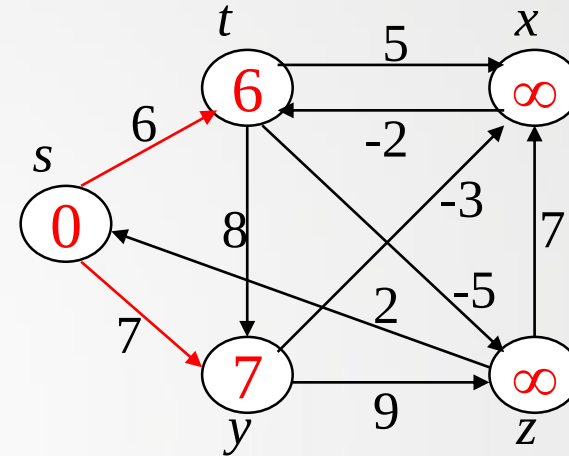
**3   for each edge  $(u, v) \in E$  do**

**4     Relax( $u, v, w$ )**

**5 for each edge  $(u, v) \in E$  do**

**6   if  $d[v] > d[u] + w(u, v)$  then return False**

**7 return True**



**BellmanFord( $G, w, s$ )**

**1 InitializeSingleSource( $G, s$ )**

**2 for  $i \leftarrow 1$  to  $|V| - 1$  do**

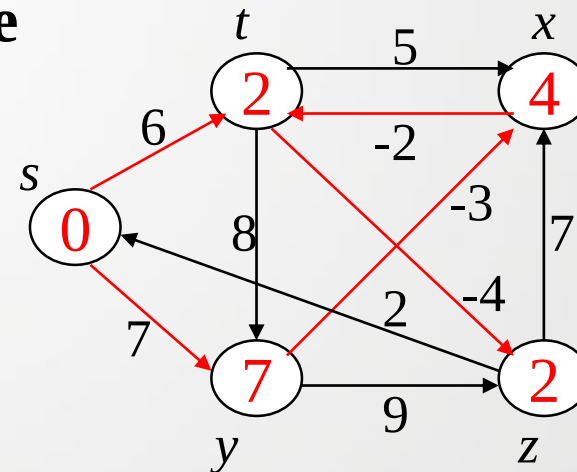
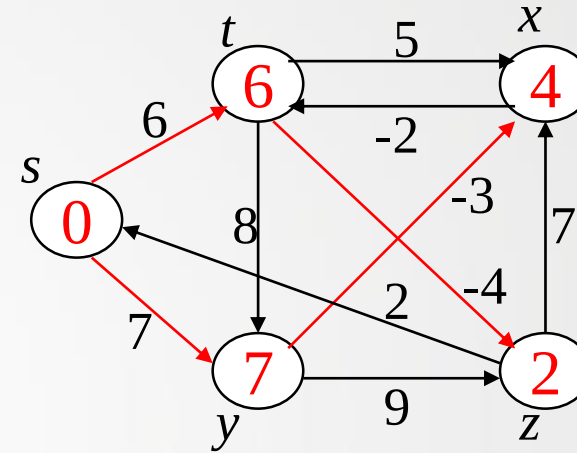
**3   for each edge  $(u, v) \in E$  do**

**4     Relax( $u, v, w$ )**

**5 for each edge  $(u, v) \in E$  do**

**6   if  $d[v] > d[u] + w(u, v)$  then return False**

**7 return True**





**BellmanFord( $G, w, s$ )**

**1 InitializeSingleSource( $G, s$ )**

**2 for  $i \leftarrow 1$  to  $|V| - 1$  do**

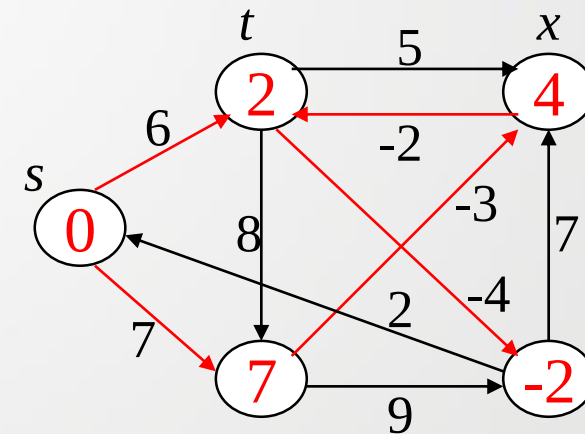
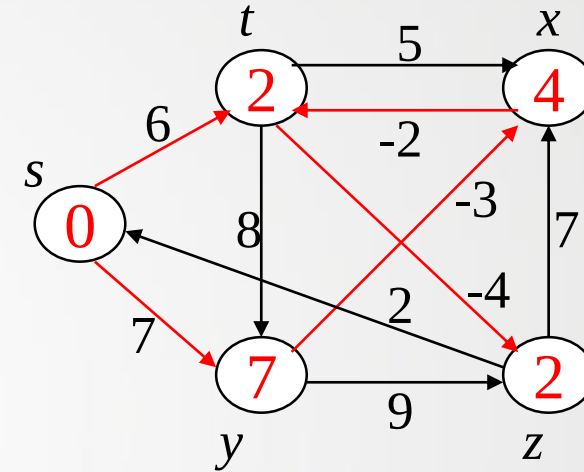
**3   for each edge  $(u, v) \in E$  do**

**4     Relax( $u, v, w$ )**

**5 for each edge  $(u, v) \in E$  do**

**6   if  $d[v] > d[u] + w(u, v)$  then return False**

**7 return True**



**BellmanFord( $G, w, s$ )**

**1 InitializeSingleSource( $G, s$ )**

**2 for  $i \leftarrow 1$  to  $|V| - 1$  do**

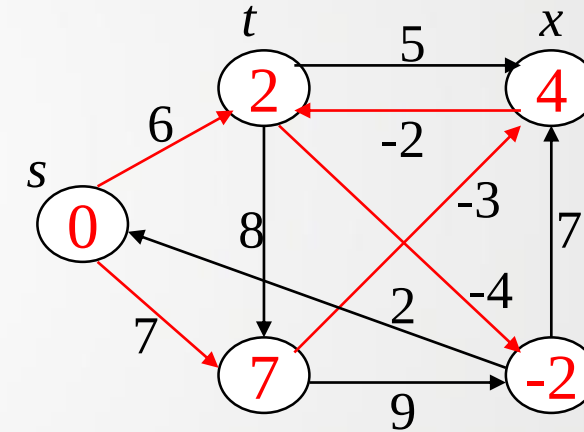
**3   for each edge  $(u, v) \in E$  do**

**4     Relax( $u, v, w$ )**

**5 for each edge  $(u, v) \in E$  do**

**6   if  $d[v] > d[u] + w(u, v)$  then return False**

**7 return True**



**TRUE**



# 一、单个源点的最短路径问题

---

## 2. Dijkstra算法

- 给定一个带权有向图  $G = (V, E)$ ，并且此图上的权值非负，对于图的单个源点的最短路径问题，我们可以用Dijkstra算法求解。
- Dijkstra 算法维持一个集合，从源点  $s$  到集合  $S$  中每个点的最短路径都已确定；这个算法反复选取中具有最短路径估计的点  $u \in V - S$ ，然后把  $u$  加入  $S$ ，并对所有与相邻的边进行松弛。

Dijkstra( $G, w, s$ )

- 1 InitializeSingleSource( $G, s$ )
- 2  $S \leftarrow \emptyset$
- 3  $Q \leftarrow V$
- 4 **while**  $Q \neq \emptyset$  **do**
- 5      $u \leftarrow \text{ExtractMin}(Q)$
- 6      $S \leftarrow S \cup \{u\}$
- 7     **for** each vertex  $v \in \text{Adj}[u]$  **do**
- 8         Relax( $u, v, w$ )

**The running time  $O(|V|^2)$**

# An example

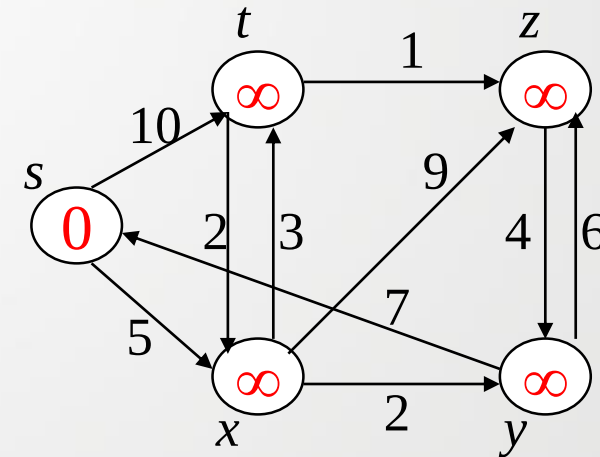
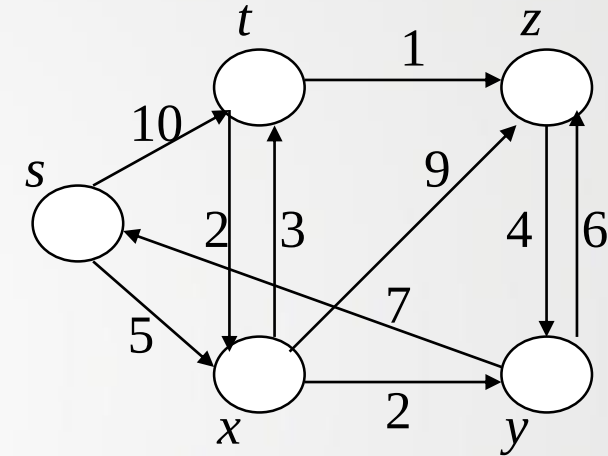
Dijkstra( $G, w, s$ )

- 1 InitializeSingleSource( $G, s$ )
- 2  $S \leftarrow \emptyset$
- 3  $Q \leftarrow V$
- 4 **while**  $Q \neq \emptyset$  **do**
- 5      $u \leftarrow \text{ExtractMin}(Q)$
- 6      $S \leftarrow S \cup \{u\}$
- 7     **for** each vertex  $v \in \text{Adj}[u]$  **do**
- 8         Relax( $u, v, w$ )

$S = \{\}$

$Q$

|     |          |          |          |          |
|-----|----------|----------|----------|----------|
| $s$ | $t$      | $z$      | $x$      | $y$      |
| 0   | $\infty$ | $\infty$ | $\infty$ | $\infty$ |



Dijkstra( $G, w, s$ )

1 InitializeSingleSource( $G, s$ )

2  $S \leftarrow \emptyset$

3  $Q \leftarrow V$

4 **while**  $Q \neq \emptyset$  **do**

5      $u \leftarrow \text{ExtractMin}(Q)$

6      $S \leftarrow S \cup \{u\}$

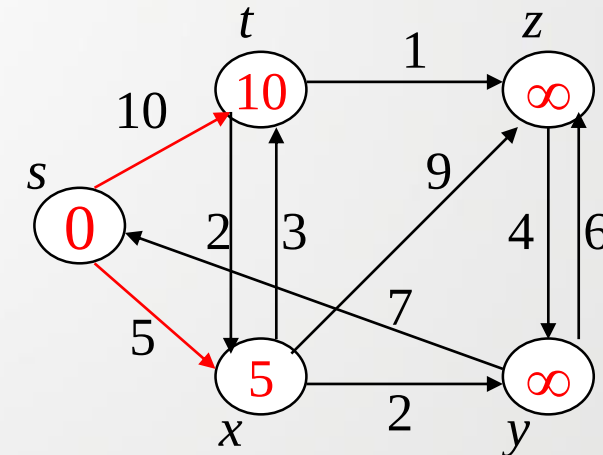
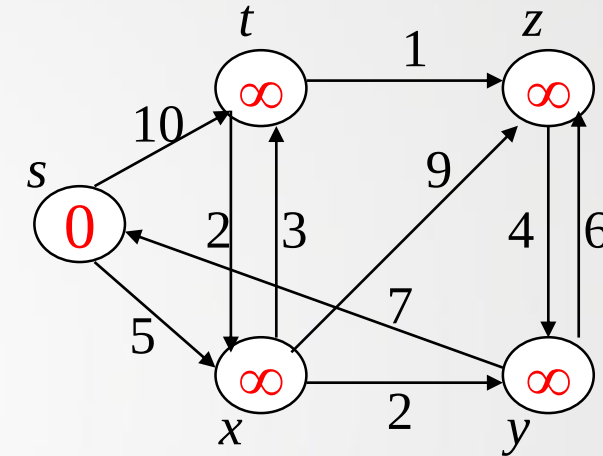
7     **for** each vertex  $v \in \text{Adj}[u]$  **do**

8         Relax( $u, v, w$ )

$Q$

|     |               |          |              |          |
|-----|---------------|----------|--------------|----------|
| $s$ | $t$           | $z$      | $x$          | $y$      |
| 0   | <del>10</del> | $\infty$ | <del>5</del> | $\infty$ |

$S = \{s\}$



Dijkstra( $G, w, s$ )

1 InitializeSingleSource( $G, s$ )

2  $S \leftarrow \emptyset$

3  $Q \leftarrow V$

4 **while**  $Q \neq \emptyset$  **do**

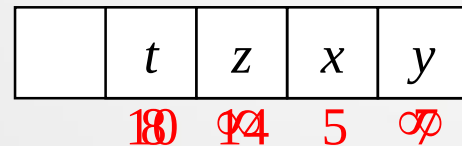
5      $u \leftarrow \text{ExtractMin}(Q)$

6      $S \leftarrow S \cup \{u\}$

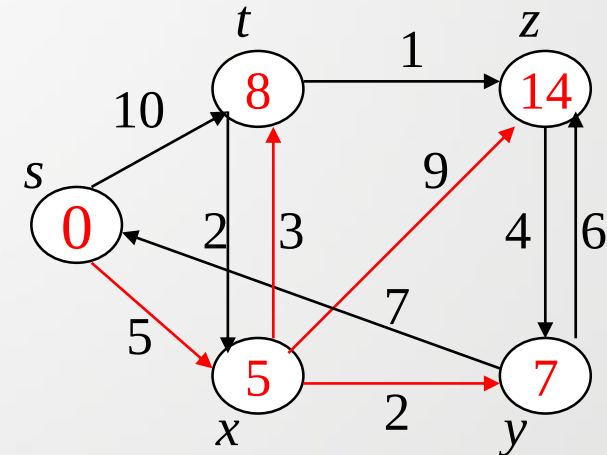
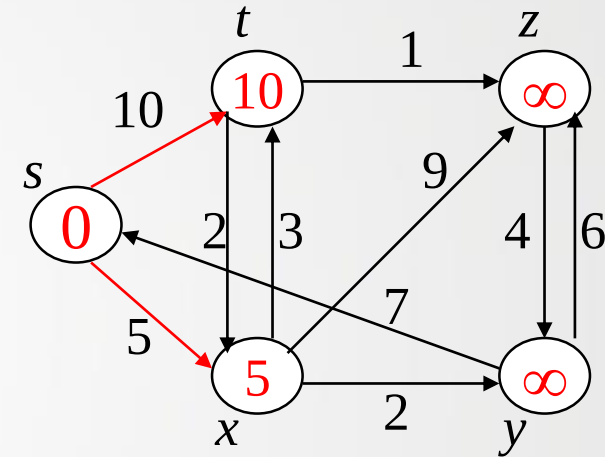
7     **for** each vertex  $v \in \text{Adj}[u]$  **do**

8         Relax( $u, v, w$ )

$Q$



$S = \{s, x\}$



Dijkstra( $G, w, s$ )

1 InitializeSingleSource( $G, s$ )

2  $S \leftarrow \emptyset$

3  $Q \leftarrow V$

4 **while**  $Q \neq \emptyset$  **do**

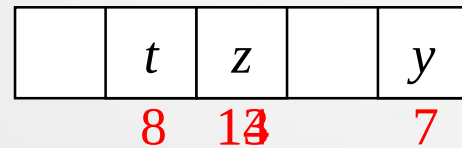
5      $u \leftarrow \text{ExtractMin}(Q)$

6      $S \leftarrow S \cup \{u\}$

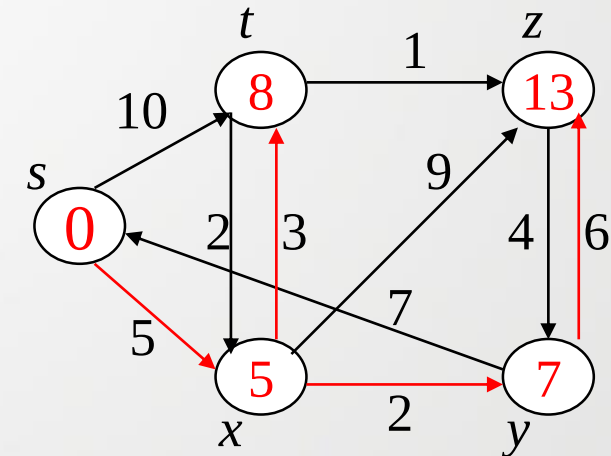
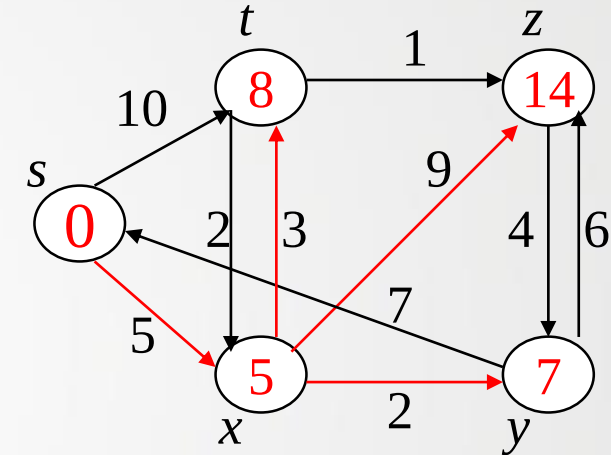
7     **for** each vertex  $v \in \text{Adj}[u]$  **do**

8         Relax( $u, v, w$ )

$Q$



$S = \{s, x, y\}$





Dijkstra( $G, w, s$ )

1 InitializeSingleSource( $G, s$ )

2  $S \leftarrow \emptyset$

3  $Q \leftarrow V$

4 **while**  $Q \neq \emptyset$  **do**

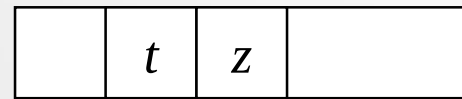
5      $u \leftarrow \text{ExtractMin}(Q)$

6      $S \leftarrow S \cup \{u\}$

7     **for** each vertex  $v \in \text{Adj}[u]$  **do**

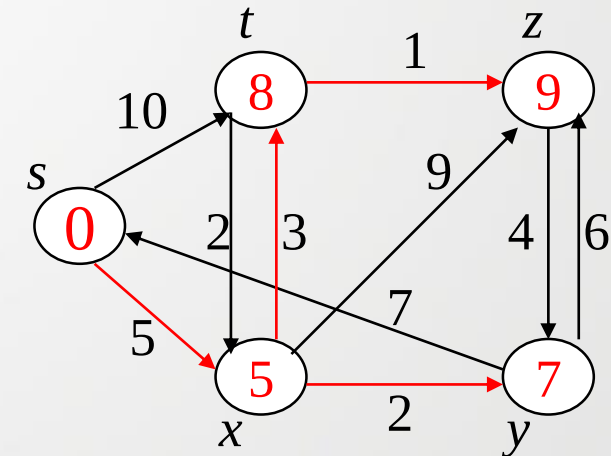
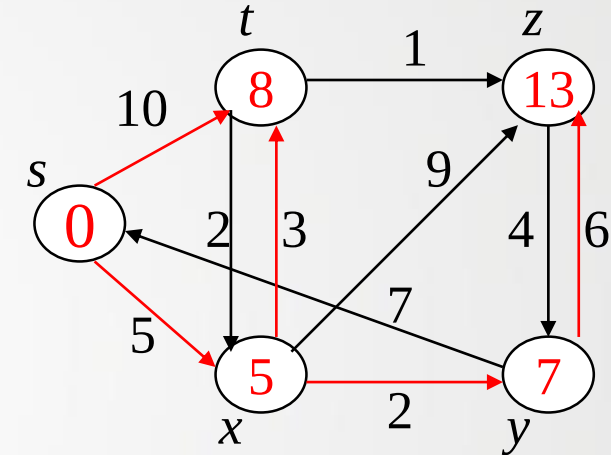
8         Relax( $u, v, w$ )

$Q$



8    93

$S = \{s, x, y, t\}$



Dijkstra( $G, w, s$ )

1 InitializeSingleSource( $G, s$ )

2  $S \leftarrow \emptyset$

3  $Q \leftarrow V$

4 **while**  $Q \neq \emptyset$  **do**

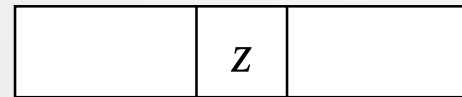
5      $u \leftarrow \text{ExtractMin}(Q)$

6      $S \leftarrow S \cup \{u\}$

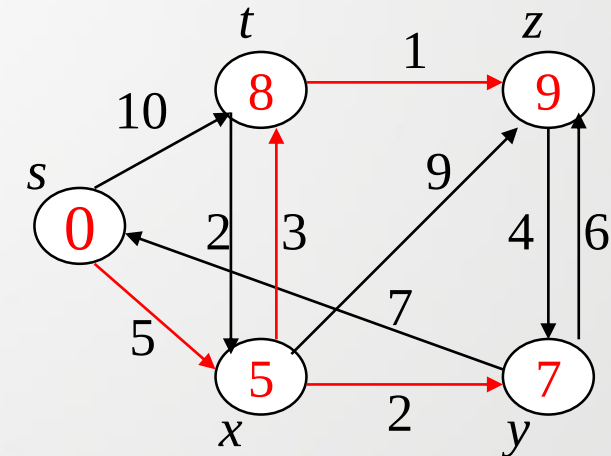
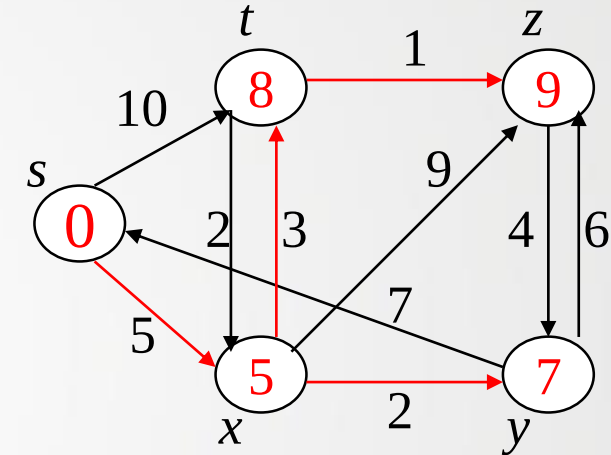
7     **for** each vertex  $v \in \text{Adj}[u]$  **do**

8         Relax( $u, v, w$ )

$Q$



$S = \{s, x, y, t, z\}$





# Dijkstra算法的正确性

## 定理8.8

- 对带非负权函数，源点为 $s$ 的有向图运用Dijkstra算法，算法终止时，对所有 $u \in V$ 有 $d[u] = \delta(s, u)$ 。

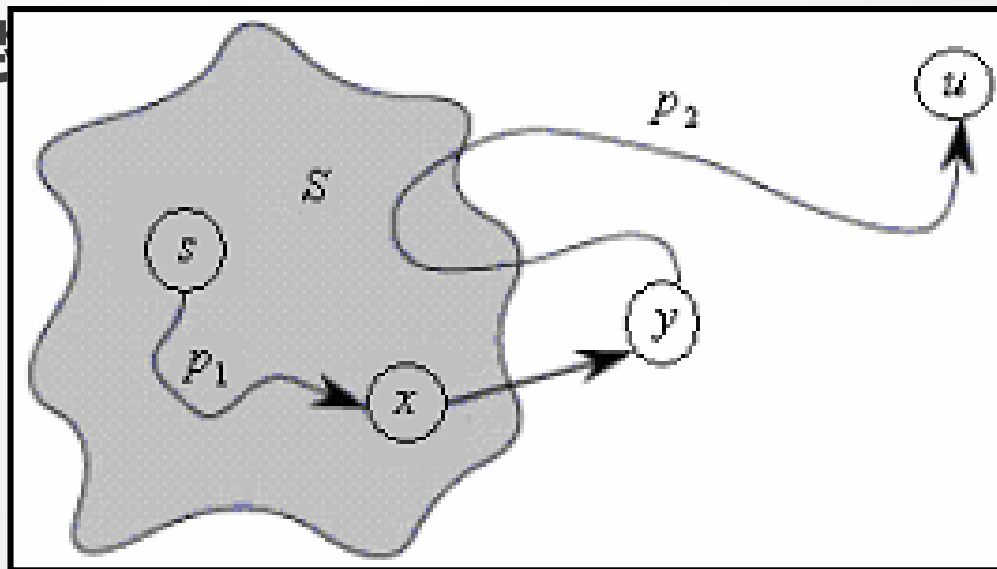
**证明：**考虑循环不变量：

在4~8行while循环的每一次迭代开始之前，对任意的 $v \in S$ ，有 $d[v] = \delta(s, v)$ 。

**初始步：**当while循环开始执行之前， $S = \emptyset$ ，因此循环不变量为真。

**归纳步：**我们希望在每一次迭代中，对于加入集合 $S$ 的顶点 $u$ 有 $d[u] = \delta(s, u)$ 。为了证明此结论，我们可以利用反证法。为了导出矛盾，令 $u$ 为迭代过程中加入集合 $S$ 的顶点中第一个满足 $d[u] \neq \delta(s, u)$ 的顶点。

- 从  $s$  到  $u$  肯定存在最短路径  $p$ ，如果没有路径的话，则有  $d[u] = \delta(s, u) = \infty$ ，与假设矛盾。在把  $u$  加入  $S$  之前，路径  $p$  连接一个顶点  $s \in S$ ，到另一个集合  $V - S$  中的点  $u$ 。如果路径  $p$  除了  $u$  之外，不含  $V - S$  中其他顶点，则由收敛性质，可得  $d[u] = \delta(s, u)$ ，因此不妨设沿着从  $s$  到  $u$  的路径  $p$  中，第一个属于  $V - S$  的顶点为  $y$ ，并设  $x \in S$  为距



8.12所示。

- 当 $u$ 被加入到 $S$ 时, 由于 $d[x] = \delta(s, x)$ , 而 $y$ 是路径 $p$ 中紧挨着顶点 $x$ 的点, 根据最短路径的收敛性质, 我们有 $d[y] = \delta(s, y)$ 。
- 现在, 为了得到一个与假设 $d[u] \neq \delta(s, u)$ 矛盾的结果, 我们只须证明 $d[u] = \delta(s, u)$ 。因为在从 $s$ 到 $u$ 的最短路径中,  $y$ 在 $u$ 之前出现, 并且所有边非负, 我们有 $\delta(s, y) \leq \delta(s, u)$ , 并且根据最短路径的上界性质 $\delta(s, u) \leq d[u]$ , 我们有 $d[y] = \delta(s, y) \leq \delta(s, u) \leq d[u]$ 。

但是由于当 $u$ 被选中时(算法第5行),  $u$  和  $y$  都属于  $V - S$  , 故有  $d[u] \leq d[y]$

根据上面两式, 我们有  $d[y] = \delta(s, y) = \delta(s, u) = d[u]$ .

即  $\delta(s, u) = d[u]$ , 这与我们选择顶点 $u$ 使得 $d[u] \neq \delta(s, u)$ 矛盾。因此我们断定在 $u$ 被加入 $S$ 后, 肯定有 $d[u] = \delta(s, u)$ , , 并且在之后的每一次迭代中都保持成立。

**终止步:** 当Dijkstra算法终止时, 队列 $Q = \emptyset$ 与循环开始前 $Q = V - S$ 一起, 暗示着 $S = V$ 。因此, 对所有点 $u \in V$ 有 $d[u] = \delta(s, u)$ 。

谢谢大家

